



DEAKIN UNIVERSITY

SIT314 Software Architecture and Scalability for Internet-of-Things

TRIMESTER 2, 2026

FACULTY OF SCIENCE, ENGINEERING AND BUILT ENVIRONMENT
SCHOOL OF ENGINEERING

**BMOAS Further Revision Report
Architecture and Measured Scalability**

Peter William Davidoff

What changed in BMOAS

BMOAS simulates a service that measures which media sources a panel of people encounters. Meters send observations, and the service produces counts by media source, region and reporting period. These counts provide a basis for audience ratings. The project uses simulated meters and observations. A commercial ratings service would also need methods for selecting participants and estimating the wider audience.

The tutor identified two problems: automatic scaling reacted too slowly, and delays in Node-RED and validation limited the whole service. I revised the processing stages, tested the changes and then made further changes to aggregation. The final trials met the original timing targets and retained the correct ratings. The test at 1,000 observations per second lasted two minutes, with capacity prepared before it began.

The requirements that guided the changes

The service must check observations, retain them during disconnection and recognise repeated deliveries. Each valid observation must be stored once, and each participant must be counted once within a reporting group. Invalid messages are separated for review. The reporting API requires a token. These functional requirements were kept while reducing processing delay.

Attempt	What changed
First revision	Reduced repeated database checks during validation, sent queue messages in groups and allowed all three processing stages to scale.
Second revision	Processed up to ten observations in each database transaction and prepared four tasks at each stage.
Final revision	Divided each ratings counter into 64 smaller groups. Reporting adds these groups together to produce the same result.

The service still follows the planned path: Node.js meters, MQTT over TLS, AWS IoT Core, SQS queues, Node-RED, validation, MongoDB Atlas, aggregation and the reporting API. AWS and Atlas services run in Sydney. A queue holds messages until the next stage can process them, allowing each stage to work at its own pace.

How the final change works

A calculation based on the participant identifier assigns each person to a consistent storage group. This spreads updates across more records instead of making many workers update the same counter. The database saves related changes together in a transaction, so they either succeed together or are rolled back. Repeated deliveries leave the ratings unchanged. The API adds the groups together before returning the original reporting format.

What was tested

BMOAS may need more capacity during news, sport or other expected events. Programme schedules and earlier ratings can help plan for these periods. The number of active meters and how often they report must also be considered. With a fixed fleet reporting at a fixed rate, a larger audience alone does not increase the message rate.

I added an optional timetable that raises the minimum to four tasks at 05:45 and 17:45, lowers it to two at 10:00 and to one at 23:00. It uses Australia/Sydney time. These are example times. The maximum stays at four, so automatic scaling can still respond during quieter periods. Lowering the minimum allows capacity to fall as demand falls.

A separate AWS test demonstrated this preparation. All three stages increased from one to four running tasks within 63.84 seconds of the scheduled time. Testing began once those tasks were ready. The daily timetable was switched off during the measured trials to keep their conditions consistent.

Final test	Observations and duration	Original target
Baseline	60,000 at 100/s for 600 seconds	p95 below 2 seconds
Higher demand	120,000 at 1,000/s for 120 seconds	p95 below 5 seconds
Reconnect	50,000 buffered observations plus 12,000 regular observations	Recovery within 120 seconds

p95 is the time within which 95% of completed observations were processed. Timing starts at the first MQTT publication attempt and ends when aggregation is confirmed in the database. Time spent waiting offline is excluded. Completed throughput is the number processed per second, including the time needed to finish after publication. Recovery measures that final period. Queue samples provide a separate check that no messages remain.

Each final test ran once. The earlier version processed 900,000 observations in a trial configured for 900 seconds. The final test at 1,000/s was shorter, at 120 seconds. Both the duration and available capacity must be considered when comparing the results.

Four tasks were ready at each processing stage. Each Node-RED and validation task had one vCPU and 2 GiB of memory. Each aggregation task had half a vCPU and 1 GiB. One further task served the API. CloudWatch recorded processor use, memory use and queues. Private networking, restricted permissions, stored secrets and encrypted connections protected the deployment.

So what do the results show

Attempt and test	Observations	p95	Outcome
First revision baseline	60,000	92.358 s	Above 2 s
Second revision baseline	60,000	0.228 s	Met target
Second revision at 1,000/s	900,000	242.231 s	Above 5 s
Final baseline	60,000	0.226 s	Met target
Final 1,000/s for 120 s	120,000	0.288 s	Met target
Final reconnect	62,000	0.506 s	Recovery met target

Table 1. The first revision began with one task per stage. Later trials prepared four. The two tests at 1,000/s had different durations. Reconnect is assessed against the recovery target.

The first revision helped, but delays remained too large. In matched trials where demand increased in steps, average p95 fell from 587.45 to 367.34 seconds. Completed throughput rose from 178.46 to 229.60 observations/s, and recovery fell from 528.75 to 304.97 seconds. The second revision met the baseline target, but aggregation still fell behind at higher demand.

The final version completed about 99.98 observations/s at baseline and 998.05 observations/s during the test at 1,000/s, including the final processing period. All generated identifiers and expected ratings matched. The captured logs recorded no service errors. This means the faster processing also retained the correct audience counts.

The reconnect trial processed all 62,000 observations. Its last database confirmation was 0.149 seconds after publication ended. The next sample confirmed empty queues after 28.32 seconds. Both measures were within the 120 second target. This supports recovering observations after a connection returns without leaving a long delay in the ratings.

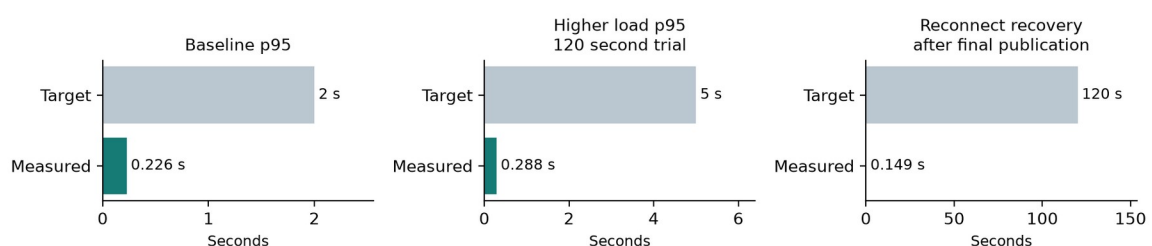


Figure 1. Final results against the original targets. The test at 1,000/s lasted 120 seconds with four tasks already running at each stage.

Now what does this mean for the design

The tests show that adding aggregation tasks alone was insufficient. Validation, message handling and shared database records also affected delay. Improving these stages together and preparing capacity helped BMOAS produce correct ratings sooner. For an expected event, capacity should be increased before it starts, allowing for the measured 63.84 seconds and a practical safety margin.

Dividing the counters spreads database updates, but creates more records and adds work when reporting combines them. Preparing tasks also uses more resources before demand arrives. These are design trade-offs: timely ratings are gained at the cost of extra capacity and a more involved reporting query.

Connection to the unit learning outcomes

ULO1 concerns the challenges of services at scale. The delays show how queues, startup time and shared records can limit performance. ULO2 concerns selecting and justifying a design against requirements. Queues separate the stages, transactions preserve ratings, and smaller counter groups reduce competition for the same records.

ULO3 concerns developing a system using available services. BMOAS combines Node.js, Node-RED, SQS, IoT Core and Atlas into a working path from meters to ratings. ULO4 concerns deploying and scaling services. The ECS deployment, scheduled increase and measured cloud trials demonstrate these activities (Deakin University, 2026).

What the evidence supports and what comes next

Verification passed 58 unit tests and two tests with a real MongoDB database. These covered repeated observations, overlapping processing, rollback, missing records, late observations and unchanged ratings. Earlier cloud checks confirmed that invalid messages were separated and the API required a valid token.

The final generator also allowed shorter gaps between publications, giving it enough capacity to reach the requested rate. The configured rates and observation format stayed the same. Appendix B records the measured rates and resource use.

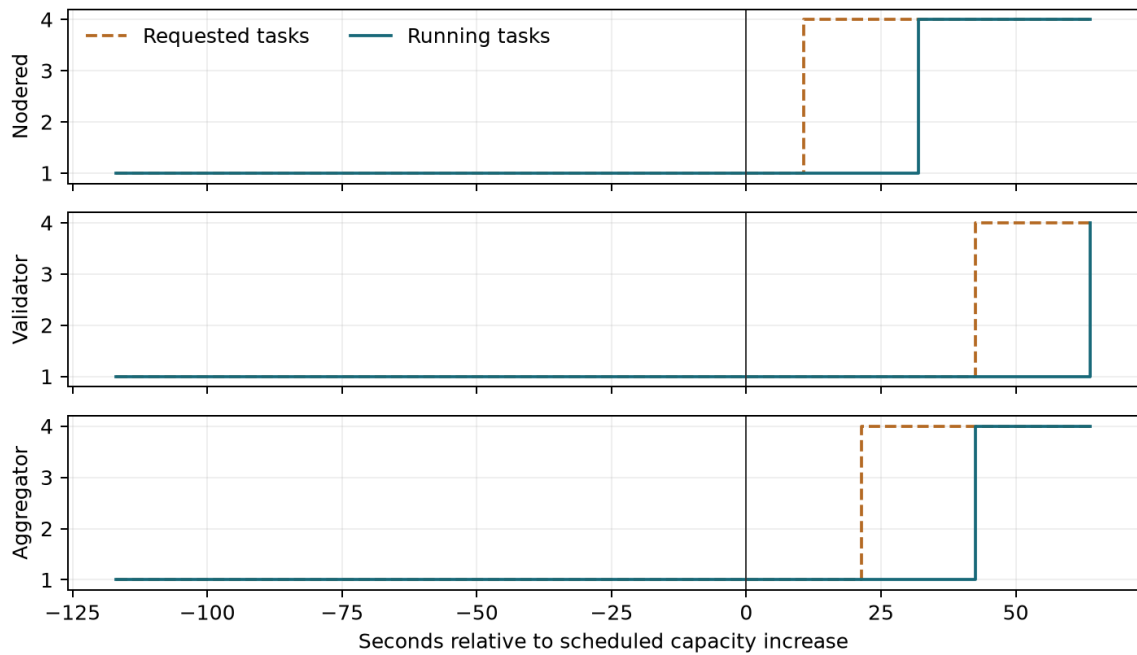
The final trials used simulated observations, one region and one Atlas tier. Each ran once, and the test at 1,000/s lasted two minutes. The next evaluation should repeat the original 15 minute test, examine unexpected demand starting with fewer tasks, and test reporting while observations arrive. These checks would show whether the gains continue for longer periods and less predictable demand. The current evidence establishes the targets under the stated conditions.

AI acknowledgement

ChatGPT and Codex generated code and assisted with configuration, tests, deployment, analysis, figures and writing. I reviewed the work and remain responsible for the submitted report, interpretation of results and explanation of the design.

Appendix A Deployment and capacity evidence

Capacity prepared before the workload



All three stages reached four running tasks after 63.84 seconds. Samples are approximately 10 seconds apart.

Figure A1. Recorded requested and running task counts during the scheduled increase. All stages were ready before the baseline workload began.

Revised stepped trial — running tasks by processing stage

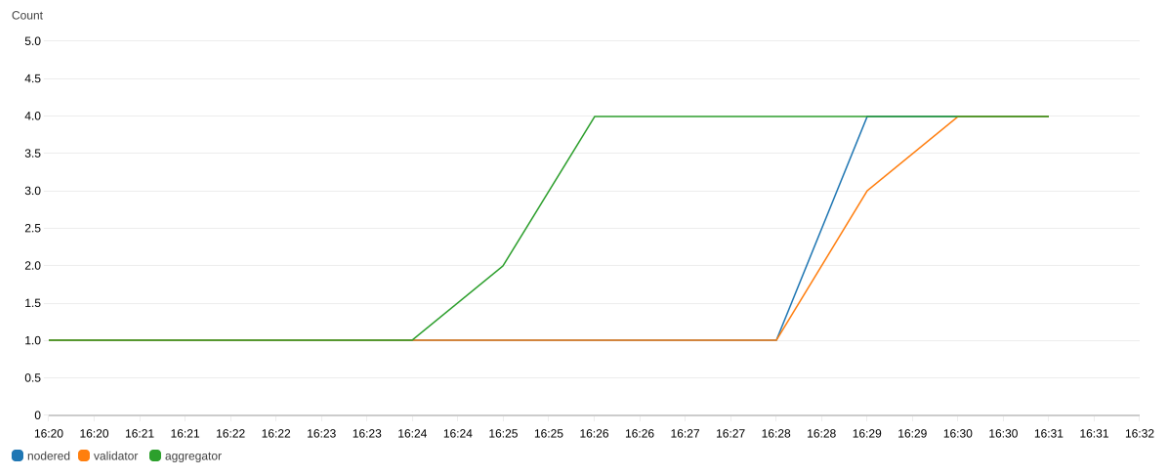


Figure A2. CloudWatch evidence from the first revision, showing capacity arriving during the stepped workload.

Appendix B Reconciled experiment evidence

Trial	Completed	p95 s	Recovery s
original-fixed-r1	60,000	245.993	250.006
original-reconnect-r1	62,000	521.172	490.620
original-stepped-r1	180,000	576.484	516.448
original-stepped-r2	180,000	598.407	541.060
revised-fixed-r1	60,000	92.358	86.634
revised-reconnect-r1	62,000	305.357	275.980
revised-stepped-r1	180,000	341.893	277.134
revised-stepped-r2	180,000	392.789	332.800
target-baseline-r1	60,000	0.228	0.232
target-scale-r1	900,000	242.231	242.618
partitioned-baseline-r1	60,000	0.226	0.123
partitioned-reconnect-r1	62,000	0.506	0.149
partitioned-scale-r1	120,000	0.288	0.215

Each trial checks that generated identifiers match completed observations and that ratings agree with a separate count of the stored observations. The second revision continued processing after collection ended, using the same software and capacity. Full results were then recovered. Queue and task samples cover only part of that recovery period.

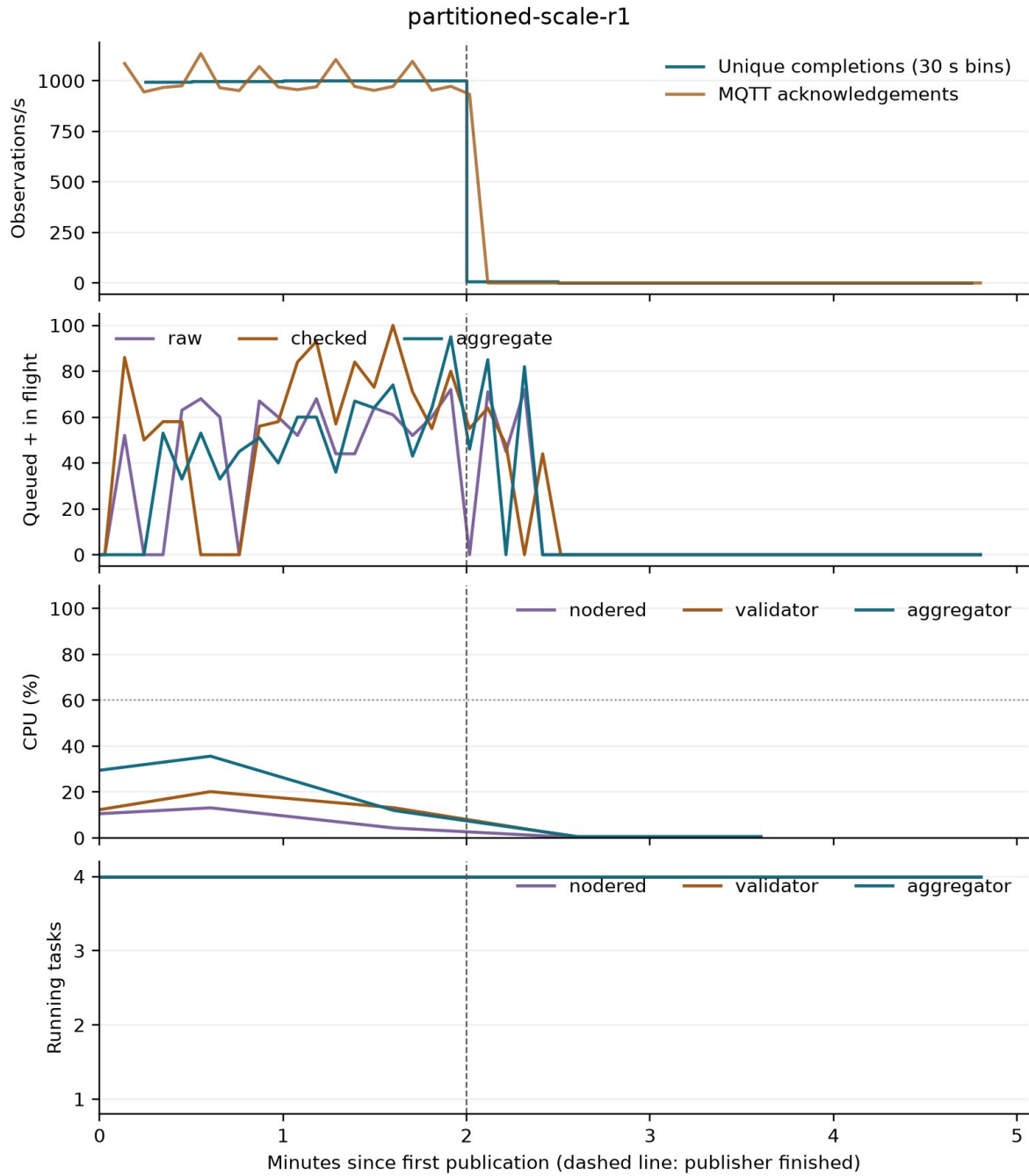


Figure B1. Final higher-load processing, queues and service measurements.

The original report and first revision report are preserved. The private repository contains source, configuration and test material at <https://github.com/pdavidoff/sit314-bmoas>. I will provide the tutor with access.

Appendix B Throughput and resource use

Final workload	Accepted /s	Completed /s	Empty queues after publication
Baseline	99.99	99.98	31.54 s
Higher load	999.81	998.05	30.79 s
Reconnect	516.58	528.08	28.32 s

Accepted throughput divides MQTT acknowledgements by the simulator interval. The reconnect simulator starts before its first publication, so its denominator is slightly longer than the completion interval. Completed throughput includes the time to the last aggregation confirmation. Queue recovery is the first sample with every observation completed and all processing queues empty. It includes the sampling interval. All three trials had exact identifiers and ratings, with no service error codes recorded in the captured logs.

Stage	Mean CPU	Mean memory	Peak sampled queue
nodered	8.73%	5.49%	72
validator	16.67%	4.88%	100
aggregator	23.80%	7.21%	95

CloudWatch means above use available one minute samples during the final higher-load processing window. Queue peaks are sampled visible plus in-flight messages. Four tasks remained running at each processing stage throughout the final trials. The reporter used one additional task. Preparing capacity allocated more resources than the original one-task starting point.

The reconnect workload included 50,000 buffered observations and 12,000 regular observations. Final commit occurred 0.149 seconds after publication ended, and the next empty-queue sample was at 28.32 seconds. Both measures were within the original 120 second recovery target.

Appendix C Implementation of the further changes

The following code is the complete new reporting method. It belongs to the MongoDB storage component. It groups stored counters by their original ratings bucket, adds observation and participant counts and removes internal partition fields before returning results. The API retains its existing limit of 500 results.

```
async reportRows(filter = {}, limit = 0) {
  const pipeline = [
    { $match: filter },
    {
      $group: {
        _id: { $ifNull: ['$logical_bucket_id', '_id'] },
        record: { $first: '$$ROOT' },
        observation_count: { $sum: '$observation_count' },
        participant_count: { $sum: '$participant_count' },
        updated_at: { $max: '$updated_at' },
      },
    },
    {
      $replaceRoot: {
        newRoot: {
          $mergeObjects: [
            '$record',
            {
              _id: '$_id',
              observation_count: '$observation_count',
              participant_count: '$participant_count',
              updated_at: '$updated_at',
            },
          ],
        },
      },
    },
    { $unset: ['logical_bucket_id', 'shard'] },
    { $sort: { window_start: 1, _id: 1 } },
  ];
  if (limit > 0) pipeline.push({ $limit: limit });
  return this.db.collection('aggregates').aggregate(pipeline).toArray();
}

async reports(filter = {}) {
  return this.reportRows(filter, 500);
}
```

Appendix C Transactional aggregation

This method belongs to the aggregation storage component. It reads the observations, assigns each participant to a stable storage group, adds new memberships and updates counters in one transaction. It then confirms completion. A repeated delivery leaves the counts unchanged. These are the implementation details used in the final cloud image.

```
async aggregateBatchOnce(
  ids,
  { windowSeconds = 60, failBeforeCommit = false, shards = 1 } = {},
) {
  const session = this.client.startSession();
  try {
    const result = await session.withTransaction(
      async () => {
        const observations = this.db.collection('observations');
        const records = await observations
          .find({ _id: { $in: ids } }, { session })
          .toArray();
        if (records.length !== ids.length) {
          throw new Error('observation_not_ready');
        }
        const outcomes = new Map();
        const buckets = new Map();
        const memberships = new Map();
        const updates = [];
        const now = new Date();
        for (const record of records) {
          if (record.completed_at) {
            outcomes.set(record._id, { duplicate: true });
            continue;
          }
          let disposition = 'counted';
          let bucketId = null;
          if (record.late) {
            disposition = 'late_review';
          } else if (record.event.media_source_id === null) {
            disposition = 'no_media';
          } else {
            const bucket = bucketFor(record, windowSeconds);
            bucketId = bucket._id;
            if (
              !Number.isSafeInteger(shards) ||
              shards < 1 ||
              shards > 256
            ) {
              throw new Error('invalid_aggregation_shards');
            }
            let storageId = bucketId;
            if (shards > 1) {
              const shard =
                parseInt(hash(record.event.panel_id).slice(0, 8), 16) %
                shards;
              storageId = hash([bucketId, shard]);
              bucket.logical_bucket_id = bucketId;
              bucket.shard = shard;
              bucket._id = storageId;
            }
            if (!buckets.has(storageId)) {
              buckets.set(storageId, {
                bucket,
                expiresAt: record.expires_at,
                observations: 0,
                participants: 0,
              });
            }
            buckets.get(storageId).observations++;
          }
        }
      }
    );
  }
}
```

```

const membershipId = hash([
  bucketId,
  record.event.panel_id,
]);
if (!memberships.has(membershipId)) {
  memberships.set(membershipId, {
    bucketId,
    storageId,
    panelId: record.event.panel_id,
    expiresAt: record.expires_at,
  });
}
}
updates.push({
  updateOne: {
    filter: { _id: record._id },
    update: {
      $set: {
        completed_at: now,
        disposition,
        bucket_id: bucketId,
        latency_ms: now - record.published_at,
      },
    },
  },
});
outcomes.set(record._id, { duplicate: false, disposition });
}
// One membership write per distinct person and bucket in this
// batch.
// The transaction makes concurrent batches retry before counts
// can diverge.
const memberEntries = [...memberships.entries()];
if (memberEntries.length) {
  const writes = memberEntries.map(([id, member]) => ({
    updateOne: {
      filter: { _id: id },
      update: {
        $setOnInsert: {
          bucket_id: member.bucketId,
          panel_id: member.panelId,
          expires_at: member.expiresAt,
        },
      },
      upsert: true,
    },
  }));
  const inserted = await this.db
    .collection('memberships')
    .bulkWrite(writes, { session });
  for (const index of Object.keys(inserted.upsertedIds)) {
    const member = memberEntries[Number(index)][1];
    buckets.get(member.storageId).participants++;
  }
}
if (buckets.size) {
  const writes = [...buckets.values()].map((group) => ({
    updateOne: {
      filter: { _id: group.bucket._id },
      update: {
        $setOnInsert: {
          ...group.bucket,
          expires_at: group.expiresAt,
        },
        $inc: {
          observation_count: group.observations,
          participant_count: group.participants,
        },
        $set: { updated_at: now },
      },
    },
  }));

```

```

        },
        upsert: true,
    },
    }));
    await this.db
        .collection('aggregates')
        .bulkWrite(writes, { session });
    }
    if (updates.length) {
        await observations.bulkWrite(updates, { session });
    }
    if (failBeforeCommit) {
        throw new Error('injected_transaction_failure');
    }
    return outcomes;
},
{
    readConcern: { level: 'snapshot' },
    writeConcern: { w: 'majority' },
    maxCommitTimeMS: 10000,
},
);
// Confirmation is written only after the whole transaction commits.
// Redelivery repairs an interrupted confirmation without increasing
// ratings.
const confirmed = new Date();
await this.db
    .collection('observations')
    .updateMany(
        { _id: { $in: ids }, confirmed_at: { $exists: false } },
        [
            {
                $set: {
                    confirmed_at: confirmed,
                    processing_latency_ms: {
                        $subtract: [confirmed, '$published_at'],
                    },
                },
            },
        ],
    ),
);
return result;
} finally {
    await session.endSession();
}
}

```

Appendix D CloudWatch and configuration

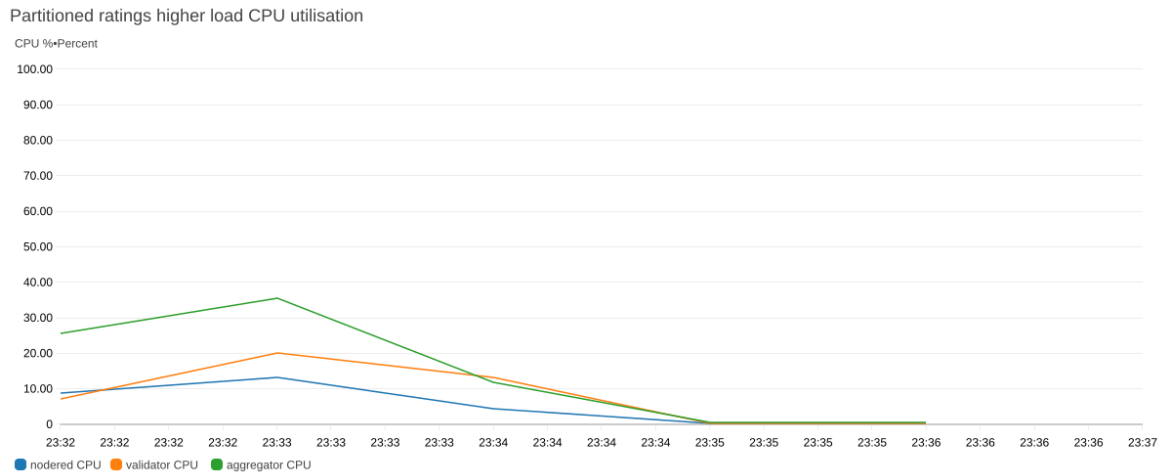


Figure D1. Graph rendered by AWS CloudWatch from the final higher-load trial. Times are UTC. The saved widget configuration identifies the services, interval and metrics.

Capacity configuration

The infrastructure definition provides optional morning, daytime, evening and overnight schedules. Each scheduled action changes the minimum capacity while retaining a maximum of four. Australia/Sydney time accounts for daylight saving. Recurring schedules default to disabled for controlled experiments. The demonstrated one-off action used the same AWS Application Auto Scaling mechanism.

Appendix D Scheduled capacity definition

This excerpt is the scheduled action configuration from the aggregation scaling target. Node-RED and validation use the same actions. The condition enables the timetable only when requested. Schedule parameters and the time zone are supplied by the infrastructure template. The deployment used four tasks per processing stage for the measured trials.

```
{
  "Fn::If": [
    "AudienceSchedule",
    [
      {
        "ScheduledActionName": "audience-morning-news",
        "Schedule": {"Ref": "MorningCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 4,
          "MaxCapacity": 4
        }
      },
      {
        "ScheduledActionName": "audience-daytime",
        "Schedule": {"Ref": "DaytimeCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 2,
          "MaxCapacity": 4
        }
      },
      {
        "ScheduledActionName": "audience-evening-news-sport",
        "Schedule": {"Ref": "EveningCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 4,
          "MaxCapacity": 4
        }
      },
      {
        "ScheduledActionName": "audience-overnight",
        "Schedule": {"Ref": "OvernightCapacitySchedule"},
        "Timezone": {"Ref": "CapacityTimezone"},
        "ScalableTargetAction": {
          "MinCapacity": 1,
          "MaxCapacity": 4
        }
      }
    ],
    {"Ref": "AWS::NoValue"}
  ]
}
```

References

Deakin University (2026), SIT314 Software Architecture and Scalability for Internet-of-Things, Unit Guide, Trimester 2, Learning outcomes, p. 3.

Amazon Web Services (2026), How scheduled scaling for Application Auto Scaling works, AWS documentation, <https://docs.aws.amazon.com/autoscaling/application/userguide/scheduled-scaling-policy-overview.html>.

Amazon Web Services (2026), AWS::ApplicationAutoScaling::ScalableTarget ScheduledAction, AWS CloudFormation documentation, <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-applicationautoscaling-scalabletarget-scheduledaction.html>.